

Algorithm

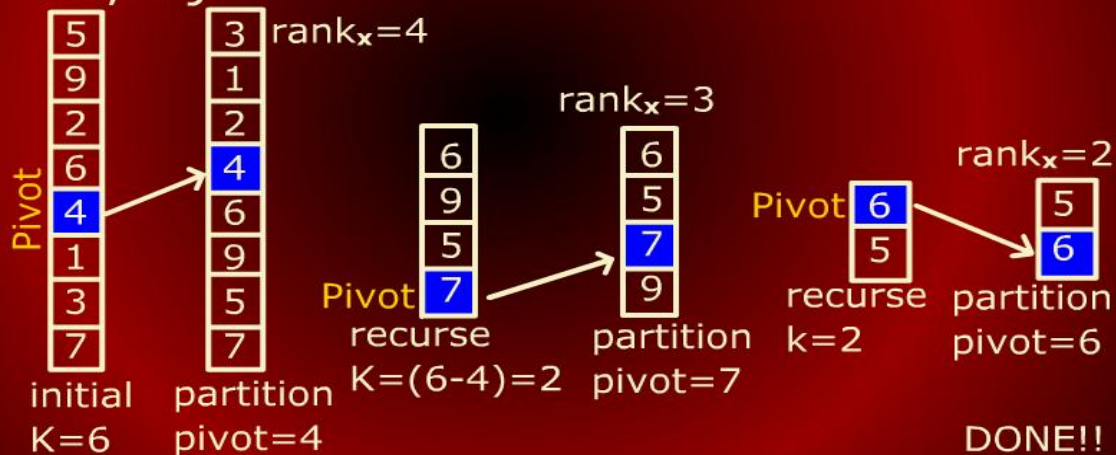
Lecture #11

Syed Hamed Raza

Select Algorithm

Select Algorithm

- Example: select the 6th smallest element of the set $\{5, 9, 2, 6, 4, 1, 3, 7\}$



Median and Selection

Medians and Selection

- We will discuss how to choose a pivot and the partitioning later.
- For the moment, we will assume that they both take $\Theta(n)$ time.
- How many elements do we eliminate in each time?

Median and Selection

Medians and Selection

- If x is the largest or the smallest then we may only succeed in eliminating one element.

5, 9, 2, 6, 4, **1**, 3, 7 pivot is 1

1, 5, 9, 2, 6, 4, 3, 7 after partition

- Ideally, x should have a rank that is neither too large or too small.

Analysis of Selection

Analysis of Selection

- Suppose we are able to choose a pivot that causes exactly half of the array to be eliminated in each phase.
- This means that we recurse on the remaining $n/2$ elements.
- This leads to the following recurrence:

$$T(n) = \begin{cases} 1 & \text{if } n = 1, \\ T(n/2) + n & \text{otherwise} \end{cases}$$

Analysis of Selection

Analysis of Selection

If we expand this recurrence, we get

$$T(n) = n + \frac{n}{2} + \frac{n}{4} + \dots$$

$$\leq \sum_{i=0}^{\infty} \frac{n}{2^i}$$

$$= n \sum_{i=0}^{\infty} \frac{1}{2^i}$$

Analysis of Selection

Analysis of Selection

- Recall the formula for infinite geometric series; for any $|c| < 1$,

$$\sum_{i=0}^{\infty} c^i = \frac{1}{1 - c}$$

- Using this we have

$$T(n) \leq 2n \in \Theta(n)$$

Analysis of Selection

Analysis of Selection

- Let's think about how we ended up with a $\Theta(n)$ algorithm for selection.
- Normally, a $\Theta(n)$ time algorithm would make a single or perhaps a constant number of passes of the data set.
- In this algorithm, we make a number of passes. In fact it could be as many as $\log n$.

Analysis of Selection

Analysis of Selection

- However, because we eliminate a constant fraction of the array with each phase, we get the convergent geometric series in the analysis.
- This shows that the total running time is indeed *linear* in n .
- This lesson is well worth remembering.
- It is often possible to achieve linear running times in ways that you would not expect.

Sorting

Sorting

Design and Analysis of Algorithm

Algorithm

Lecture #11

Sohail Aslam

Select Algorithm

Select Algorithm

```
SELECT( array A, int p, int r, int k)
1 if (p = r)
2   then return A[p]
3 else x ← CHOOSE PIVOT(A, p, r)
4   q ← PARTITION(A, p, r, x)
5   rank_x ← q - p + 1
6   if k = rank_x
7     then return x
8   if k < rank x
```

Select Algorithm

Select Algorithm

```
5  rank_x  $\leftarrow$  q - p + 1
6  if k = rank_x
7    then return x
8  if k < rank_x
9    then return SELECT(A, p, q-1, k)
10 else return SELECT(A, q+1, r, k-q)
```

Sorting

- For the next series of lectures, we will focus on sorting.
- There are a number of reasons for sorting. Here are a few important ones.
- Procedures for sorting are parts of many large software systems.
- Design of efficient sorting algorithms is necessary to achieve overall efficiency of these systems.

Sorting

- Sorting is well studied problem from the analysis point of view.
- Sorting is one of the few problems where provable lower bounds exist on how fast we can sort.
- In sorting, we are given an array $A[1..n]$ of n numbers
- We are to reorder these elements into increasing (or decreasing) order.

Sorting

- More generally, A is an array of objects and we sort them based on one of the attributes - the key value.
- The key value need not be a number. It can be any object from a totally ordered domain.
- Totally ordered domain means that for any two elements of the domain, x and y , either $x < y$, $x = y$ or $x > y$.

Slow Sorting Algorithms

Bubble sort

- Scan the array.
- Whenever two consecutive items are found that are out of order, swap them.
- Repeat until all consecutive items are in order.

Slow Sorting Algorithms

Insertion sort

- Assume that $A[1..i - 1]$ have already been sorted.
- Insert $A[i]$ into its proper position in this sub array.
- Create this position by shifting all larger elements to the right.

Slow Sorting Algorithms

Selection sort

- Assume that $A[1 \dots i-1]$ contain the $i-1$ smallest elements in sorted order.
- Find the smallest element in $A[i..n]$
- Swap it with $A[i]$.

Slow Sorting Algorithms

- These algorithms are easy to implement.
- But they run in $\Theta(n^2)$ time in the worst case.

$n \log n$ Sorting

- We have already seen that Mergesort sorts an array of numbers in $\Theta(n \log n)$ time.
- We will study two others: *Heapsort* and *Quicksort*.

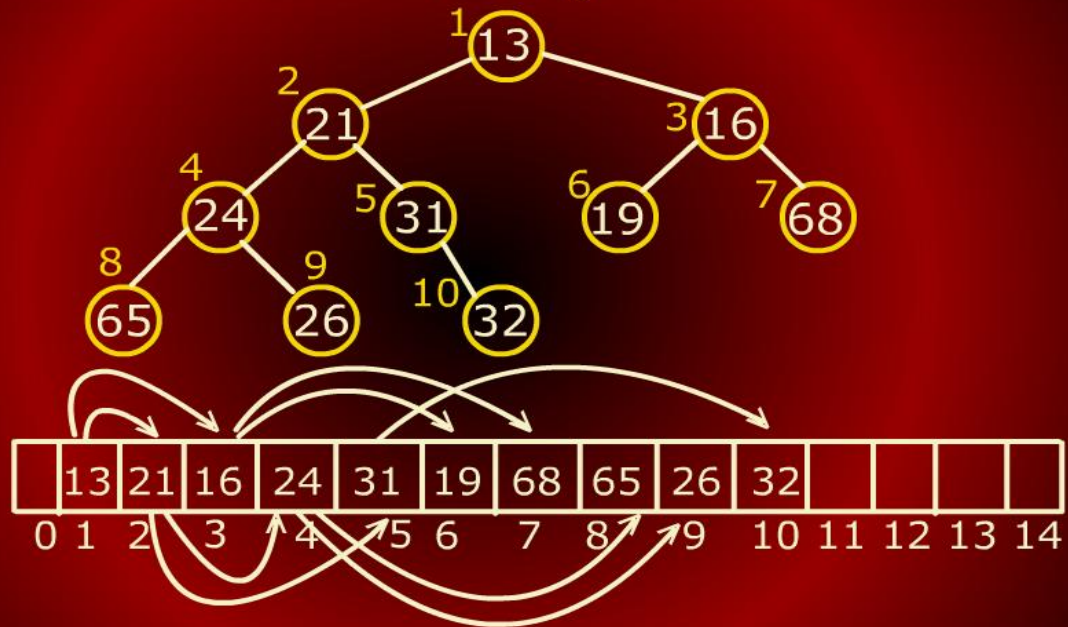
Heap

- A heap is a left-complete binary tree that conforms to the *heap order*.
- The heap order property: in a (min) heap, for every node X , the key in the parent is smaller than or equal to the key in X .
- In other words, the parent node has key smaller than or equal to both of its children nodes.

Heap

- Similarly, in a max heap, the parent has a key larger than or equal both of its children
- Thus the smallest key is in the root in a min heap; in the max heap, the largest is in the root.

Heap



Heap Sort
Quick Sort
 $O(n \log n)$

Bubble Sort
Insertion Sort
Selection Sort
 $O(n^2)$

Analysis of Selection